

Examen de Programmation Orientée Objet

Réponses

Exercice 1 - Modélisation du Transport de Marchandises

1. Classe abstraite Cargaison

Code

```
1 public abstract sealed class Cargaison
2     permits CargaisonFluviale, CargaisonRoutiere,
3         CargaisonAerienne {
4
5     protected int distance; // distance en kilom tres
6     protected int charge;   // poids total en kg
7
8     public Cargaison(int distance) {
9         this.distance = distance;
10        this.charge = 0;
11    }
12
13    public int getDistance() {
14        return distance;
15    }
16
17    public int getCharge() {
18        return charge;
19    }
20
21    protected void ajouterCharge(int poids) {
22        this.charge += poids;
23    }
24
25    // M thodes abstraites
26    public abstract void ajouter(int poids);
27    public abstract double calculerCout();
28 }
```

Réponse

La classe est déclarée `sealed` pour restreindre l'héritage uniquement aux trois classes spécifiées.

2. Interface Transportable

Code

```
1 public interface Transportable {
2     // M thode abstraite
3     void afficherDetails();
4
5     // M thode par d faut
6     default void afficherType() {
7         System.out.println("Type de cargaison");
8     }
9
10    // M thode statique
11    static void afficherInfoGenerale() {
12        System.out.println("Types de cargaisons disponibles : " +
13                            "Fluviale, Routi re, A rienne");
14    }
15 }
```

3. Classes concrètes

Code

```
1 // Classe CargaisonFluviale
2 public final class CargaisonFluviale
3     extends Cargaison implements Transportable {
4
5     public static final int CAPACITE_MAX = 30000;
6
7     public CargaisonFluviale(int distance) {
8         super(distance);
9     }
10
11     @Override
12     public void ajouter(int poids) {
13         if (charge + poids <= CAPACITE_MAX) {
14             ajouterCharge(poids);
15         } else {
16             System.out.println("Capacit  maximale d  pass e");
17         }
18     }
19
20     @Override
21     public double calculerCout() {
22         return distance * charge / 100.0;
23     }
24
25     @Override
26     public void afficherDetails() {
27         System.out.println("Cargaison Fluviale - " +
28                             "Distance: " + distance + " km, " +
29                             "Charge: " + charge + " kg");
30     }
31 }
32
33 // Classe CargaisonRoutiere
34 public final class CargaisonRoutiere
35     extends Cargaison implements Transportable {
36
37     public static final int CAPACITE_MAX = 38000;
38
39     // Constructeur et m thodes similaires...
40     // Co t calcul  : 2 * distance * charge / 100
41 }
42
43 // Classe CargaisonAerienne
44 public final class CargaisonAerienne
45     extends Cargaison implements Transportable {
46
47     public static final int CAPACITE_MAX = 80000;
48
49     // Constructeur et m thodes similaires...
50     // Co t calcul  : 4 * distance * charge / 100
51 }
```

4. Record ResultatTransport

Code

```
1 public record ResultatTransport(String typeCargaison,
2                                double coutTransport) {
3     public ResultatTransport {
4         // Conversion en minuscules comme demand
5         typeCargaison = typeCargaison.toLowerCase();
6     }
7
8     @Override
9     public String toString() {
10         return String.format("Type: %s, Co t: %.2f ",
11                               typeCargaison, coutTransport);
12     }
13 }
```

5. Classe principale Transportation

Code

```
1 import java.util.Scanner;
2
3 public class Transportation {
4     public static void main(String[] args) {
5         Scanner scanner = new Scanner(System.in);
6
7         // 1. Afficher info g n rale
8         Transportable.afficherInfoGenerale();
9
10        // 2. Cr er une cargaison
11        System.out.print("Type de cargaison : ");
12        String type = scanner.nextLine().toLowerCase();
13
14        System.out.print("Distance en km : ");
15        int distance = scanner.nextInt();
16
17        Cargaison cargaison = switch(type) {
18            case "fluviale" -> new CargaisonFluviale(distance);
19            case "routi re", "routiere" ->
20                new CargaisonRoutiere(distance);
21            case "a rienne", "aerienne" ->
22                new CargaisonAerienne(distance);
23            default -> {
24                System.out.println("Type invalide!");
25                yield null;
26            }
27        };
28
29        if (cargaison == null) return;
30
31        // 3. Ajouter les marchandises
32        int[] poids = {10000, 4000, 2000, 11500};
33        for (int p : poids) {
34            cargaison.ajouter(p);
35        }
36
37        // 4. Calculer et afficher
38        double cout = cargaison.calculerCout();
39        ((Transportable) cargaison).afficherDetails();
40
41        // 5. Utiliser le record
42        ResultatTransport resultat =
43            new ResultatTransport(type, cout);
44        System.out.println(resultat);
45
46        scanner.close();
47    }
48 }
```

Exercice 2 - Analyse de Code Java

1. Premier exemple

Code

```
1 class A {
2     public A() {
3         System.out.println("Je suis dans A");
4     }
5 }
6
7 class B extends A {
8     public B() {
9         System.out.println("B prologue");
10        super(); // ERREUR ICI
11        System.out.println("B epilogue");
12    }
13 }
14
15 public class Test1 {
16     public static void main(String[] args) {
17         A obj = new B();
18     }
19 }
```

Réponse

Erreur de compilation

Cause : L'appel à `super()` doit être la première instruction dans un constructeur.

Correction :

```
1 class B extends A {
2     public B() {
3         super(); // Doit tre en premier
4         System.out.println("B prologue");
5         System.out.println("B epilogue");
6     }
7 }
```

Sortie après correction :

```
Je suis dans A
B prologue
B epilogue
```

2. Deuxième exemple

Code

```
1 public class Test2 {
2     public static int f(Integer... x) { return 1;}
3     public static int f(long x) {return 2;}
4     public static int f(short x) {return 3;}
5     public static void main(String[] args) {
6         var val = 10;
7         System.out.println(f(val));
8     }
9 }
```

Réponse

Correct

Explication : Pour `val = 10` (type `int`), le compilateur choisit la méthode la plus spécifique : `f(long x)`.

Sortie : 2

3. Troisième exemple

Code

```
1 interface Operation {
2     void affiche();
3 }
4
5 public class Test3 {
6     public void printMessage() {
7         System.out.println("Bienvenue");
8     }
9     public static void main(String[] args) {
10         Operation obj = Test3::printMessage; // ERREUR ICI
11         obj.affiche();
12     }
13 }
```

Réponse

Erreur de compilation

Cause : `printMessage()` est une méthode d'instance, pas statique.

Correction 1 :

```
1 public static void printMessage() { // Rendre statique
2     System.out.println("Bienvenue");
3 }
```

Correction 2 :

```
1 Operation obj = new Test3()::printMessage; // Cr er instance
```

4. Quatrième exemple

Code

```
1 class X {
2     public int f(int a) { return a++;}
3 }
4 class Y extends X {
5     protected int f(int a , int b) { return a+b;}
6 }
7 public class Test4 {
8     public static void main(String[] args) {
9         X obj = new Y();
10        System.out.println(obj.f(5));
11    }
12 }
```

Réponse

Correct

Explication : `Y.f(int, int)` ne redéfinit pas `X.f(int)` (signatures différentes).

Sortie : 5

(Rappel : `a++` retourne la valeur avant incrémentation)

5. Cinquième exemple

Code

```
1 public class Test5 {
2     public static void main(String[] args) {
3         StringBuffer sb = new StringBuffer("EN");
4         sb.append("CPI2").insert(1, "XAME");
5         System.out.println(sb.capacity());
6         System.out.println(sb.length());
7         System.out.println(sb.toString());
8     }
9 }
```

Réponse

Correct

Explication :

- Initialisation : "EN" (longueur 2)
- `append("CPI2")` → "ENCPI2" (longueur 6)
- `insert(1, "XAME")` → "E" + "XAME" + "NCPI2" = "EXAMENCPI2"

Sortie :

20

10

EXAMENCPI2

6. Sixième exemple

Code

```
1 class A1 {
2     public String f(B1 obj) { return "A1 et B1"; }
3 }
4 class B1 extends A1 {
5     public String f(A1 obj) { return "B1 et A1"; }
6 }
7 interface X1 {
8     default String f(A1 obj) { return "X1 et A1"; }
9 }
10 interface Y1 extends X1 {
11     default String f(A1 obj) { return "Y1 et A1"; }
12 }
13 class C1 extends A1 implements Y1 {
14     public String f(A1 obj) { return Y1.super.f(obj); }
15 }
16
17 public class Test6 {
18     public static void main(String[] args) {
19         Y1 obj1 = new C1();
20         System.out.println(obj1 instanceof X1);
21         B1 obj2 = (B1) obj1; // ERREUR ICI
22         System.out.println(obj2.f(new A1()));
23     }
24 }
```

Réponse

Erreur d'exécution (ClassCastException)

Cause : C1 hérite de A1 et implémente Y1, mais n'a aucune relation avec B1.

Explication détaillée :

- obj1 instanceof X1 → true (car Y1 extends X1)
- (B1) obj1 → Échoue car C1 n'est pas un B1

Sortie partielle avant erreur :

true

Exception in thread "main" java.lang.ClassCastException:
class C1 cannot be cast to class B1

Correction possible :

```
1 // Remplacer la ligne problématique par :
2 A1 obj2 = (A1) obj1; // OK car C1 hérite de A1
3 System.out.println(obj2.f(new B1())); // Appelle A1.f(B1)
```

Résumé des Réponses de l'Exercice 2

Numéro	Correct/Erreur	Type d'erreur	Sortie/Correction
1	Erreur	Compilation	<code>super()</code> mal placé
2	Correct	-	Sortie : 2
3	Erreur	Compilation	Référence méthode incorrecte
4	Correct	-	Sortie : 5
5	Correct	-	20, 10, "EXAMENCPI2"
6	Erreur	Exécution	ClassCastException

Fin de l'examen